PortScan-OS

Port Scanning with OS Information and Version Detection

A Custom Attack Toolkit and Defensive Countermeasure Architecture from Scratch

Assigned Topic: 8
Topic Title: Port Scanning with OS information / version
Subsection: A2
Supervisor: **Anwarul Bashir Shuaib**
Submitted By: **Nawriz Ahmed Turjo (2105032)**
Abhishek Roy (2105033)

Core Attack Pipeline & Defensive Countermeasures

Raw Layer-2 TCP SYN Port Scanner

Stream Banner Grabbing (FTP & HTTP)

Heuristic TCP/IP OS Fingerprinting

Sliding-Window Dynamic Firewall Defense

1 Overview of the Project Idea

1.1 Problem Statement and Threat Model

Network reconnaissance determines which communication channels and service ports are reachable across a network perimeter, what software daemon versions run on those ports, and what operating system kernel manages the host. While off-the-shelf tools like Nmap automate this process, building a custom reconnaissance engine demonstrates how raw packets are serialized and parsed on Windows. Operating within an adjacent local subnet, this toolkit manually constructs Ethernet, IPv4, and TCP frames to identify reachable ports, fingerprint the host operating system, and validate exposed services within defined safety limits.

1.2 Project Scope and Five-Phase Attack Chain

The project organizes its workflow into five attack phases alongside a concurrent defensive module:

Table 1: Five-Phase Reconnaissance and Attack Architecture

Phase	Designation	Operational Focus	Engineering Mechanism
1	**TCP SYN Port Scan**	Detect Open, Closed, and Filtered ports	Raw Layer-2 frame construction via Npcap; half-open scan without completing the 3-way handshake.
2	**Banner Grabbing**	Service and version identification	Standard stream sockets querying listening daemons (passive capture for FTP, active HTTP probes).
3	**OS Fingerprinting**	Identify target host operating system	Heuristic analysis of SYN/ACK response headers: initial TTL, window size, MSS, and TCP option sequences.
4	**Vulnerability Match**	Map findings to attack surfaces	Automated mapping between extracted daemon versions and known proof-of-concept exploit vectors.
5	**Exploit Execution**	Demonstrate concrete security compromise	Automated dispatch of anonymous FTP authentication and HTTP directory path traversal payloads.
Defense	**Scan Detection & Block**	Passive detection and dynamic mitigation	Sliding-window inspection tracking SYN packet rates and destination port diversity with dynamic Windows Firewall rule generation.

1.3 Core Design Principles

- **From-Scratch Packet Construction:** All Ethernet frames, IPv4 packets, and TCP segments are assembled byte by byte in binary format. The implementation avoids high-level network construction abstractions.
- **Direct Checksum Computation:** Checksum fields for both IPv4 headers and TCP segments (with their accompanying pseudo-headers) are computed via a custom 16-bit one's complement Internet checksum implementation following RFC 1071.
- **Npcap as Hardware Interface:** The toolkit uses Npcap strictly as a low-level packet I/O driver on Windows via Python `ctypes`. All probe dispatching, frame sequencing, and response interpretation run in custom application code.
- **Strict Safety Boundaries:** The toolkit restricts scanning to private IPv4 address spaces (RFC 1918), caps targets at 50 ports per run, and enforces an inter-probe delay of at least 0.2 seconds to prevent denial of service conditions.

2 Network Topology and System Components

2.1 Laboratory Network Architecture

The experimental testbed operates within an isolated private local subnet (such as `192.168.1.0/24` on a dedicated physical switch or a host-only virtualized hypervisor segment). Both nodes share the same Layer-2 broadcast domain because the custom scanner crafts raw Ethernet frames and performs Address Resolution Protocol (ARP) queries directly to obtain target MAC addresses.

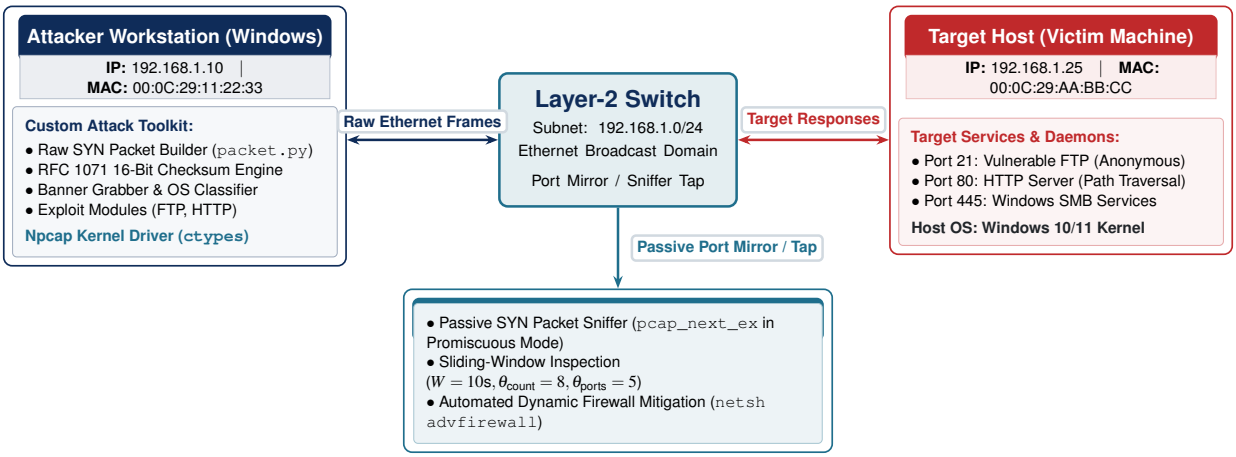


Figure 1: Laboratory Network Topology, Host Roles, and Traffic Monitoring Architecture

2.2 Component Specifications

Table 2: System Component Inventory and Operational Responsibilities

Component	Host Machine	Software Basis	Role in Evaluation
SYN Scanner	Attacker	Python / `packet.py`	Builds raw Ethernet/IPv4/TCP frames, dispatches SYN packets, and logs responses.
Banner Grabber	Attacker	Python / `banner.py`	Performs stream handshakes to capture service banners from open ports.
OS Classifier	Attacker	Python / `fingerprint.py`	Parses SYN/ACK attributes to calculate operating system probability scores.
Exploit Suite	Attacker	Python / `exploits/`	Executes targeted exploit modules against confirmed vulnerable ports.
Npcap Driver	Attacker/Victim	Npcap Native DLLs	Provides low-level kernel packet capture and raw transmission interfaces.
Vulnerable FTP	Victim	Python / `ftp_server.py`	Simulates standard RFC 959 FTP with anonymous read authorization.
Vulnerable HTTP	Victim	Python / `http_server.py`	Web server with intentional path traversal vulnerability (CVE-2021-41773 analogue).
Defense Monitor	Victim	Python / `detector.py`	Passive traffic sniffer calculating sliding-window SYN probe arrival rates.
Firewall Agent	Victim	Windows `netsh`	Injects dynamic inbound blocking rules when scanning thresholds are breached.

3 Protocol Mechanics and Attack Timing Diagrams

3.1 Original Protocol vs. SYN Port Scan Mechanics

Standard TCP (RFC 793) connections complete a full three-way handshake: SYN → SYN/ACK → ACK. Application listeners accept and log the connection only after the final ACK arrives.

In contrast, our TCP SYN scanner performs a half-open scan. It sends a single SYN probe and inspects the reply. If the port is open, the target returns a SYN/ACK. The scanner records the open state and withholds the final ACK, leaving the connection unestablished and typically unlogged by user-space applications. If the port is closed, the target kernel returns a RST packet. If a firewall drops the probe, no packet arrives before the 1.5-second timeout expires.

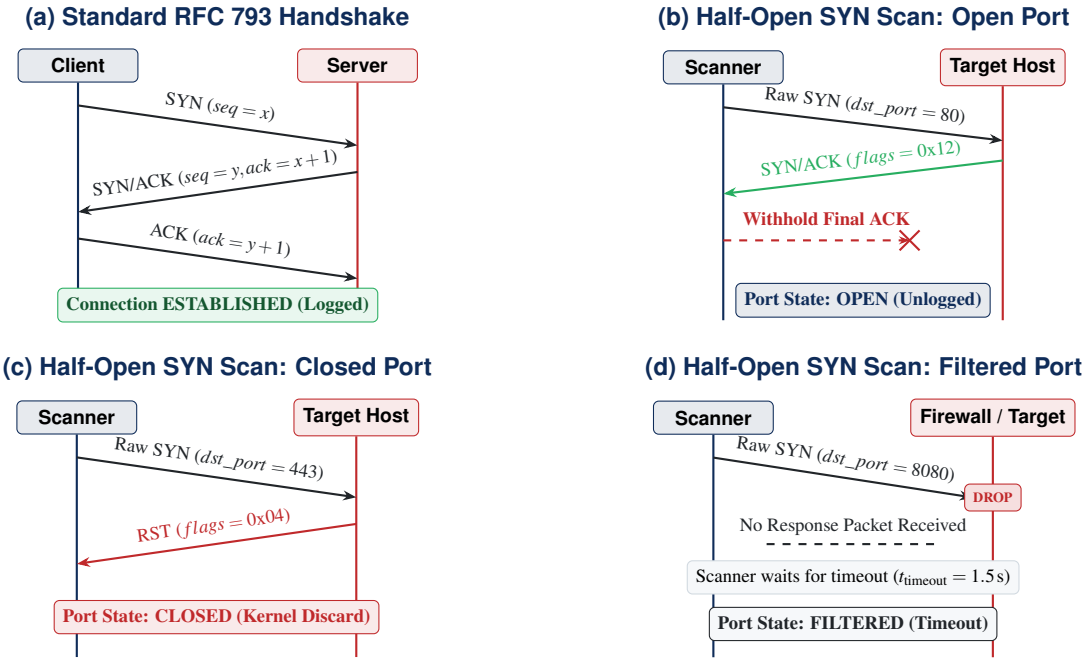


Figure 2: Protocol Timing Diagrams Comparing Standard TCP Handshake with Scanner Probe Classifications

3.2 Service Banner Grabbing Timing Sequences

Banner grabbing extracts identification strings from services running on open ports. As shown in Figure 3, banner grabbing follows two patterns depending on whether the target service is passive or active:

- **Passive Services (e.g., FTP, SMTP):** The listening daemon transmits an informational greeting banner immediately upon completion of the initial TCP 3-way handshake. The grabber establishes a standard connection, reads the incoming buffer, and terminates cleanly.
- **Active Services (HTTP):** Web daemons do not output unprompted greetings. The grabber establishes a connection, issues a minimal RFC-compliant probe (`HEAD / HTTP/1.0\r\n\r\n`), and extracts the software identification from the HTTP `Server:` header.

3.3 Targeted Exploit Execution Timing

After cataloging open ports and daemon versions, the orchestrator runs the targeted exploit modules. Figure 4 outlines the message sequences for anonymous FTP data access and HTTP path traversal.

3.4 Defense Detection and Mitigation Timeline

Figure 5 shows the defense monitor timeline. Incoming SYN packets update a sliding-window queue on the target host. When probe volume reaches 8 SYN packets across at least 5 distinct ports within 10 seconds, the monitor triggers an alert and adds an inbound firewall rule blocking the scanner's IP. The network interface drops subsequent probes before they reach application sockets.

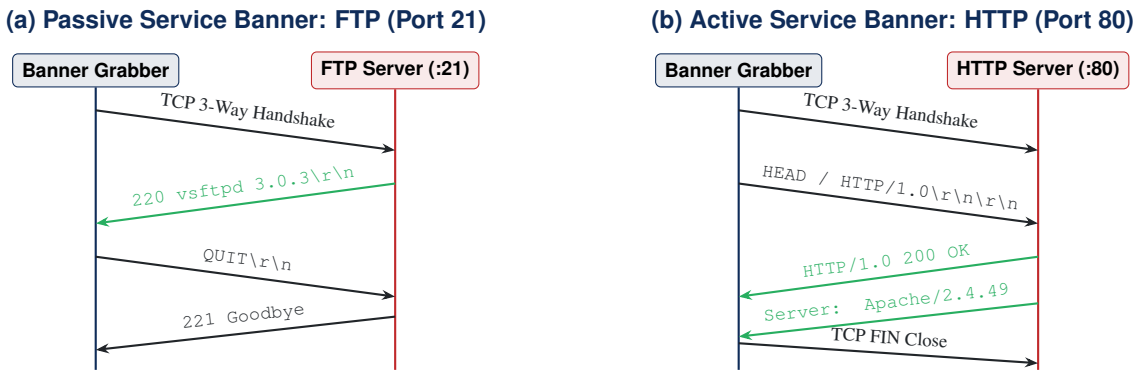


Figure 3: Timing Sequences for Passive Banner Capture and Active Application Probing

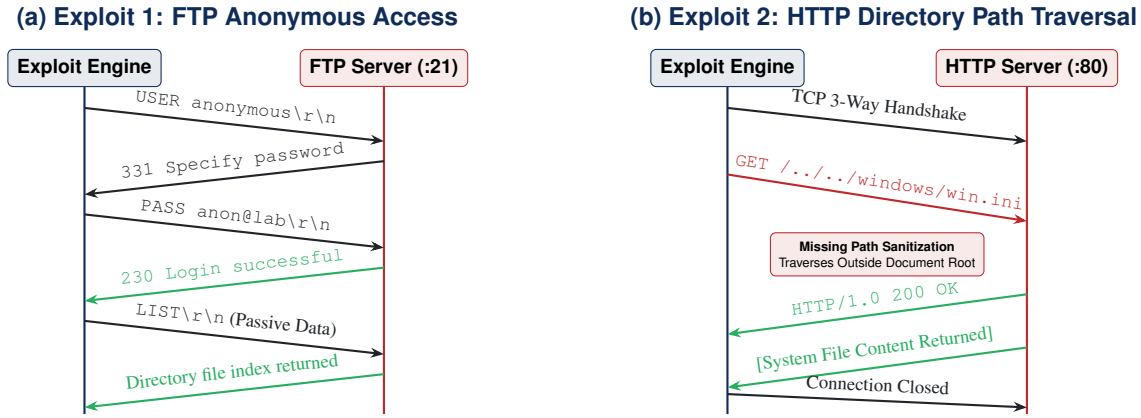


Figure 4: Protocol Interactions Demonstrating Concrete Exploit Outcomes

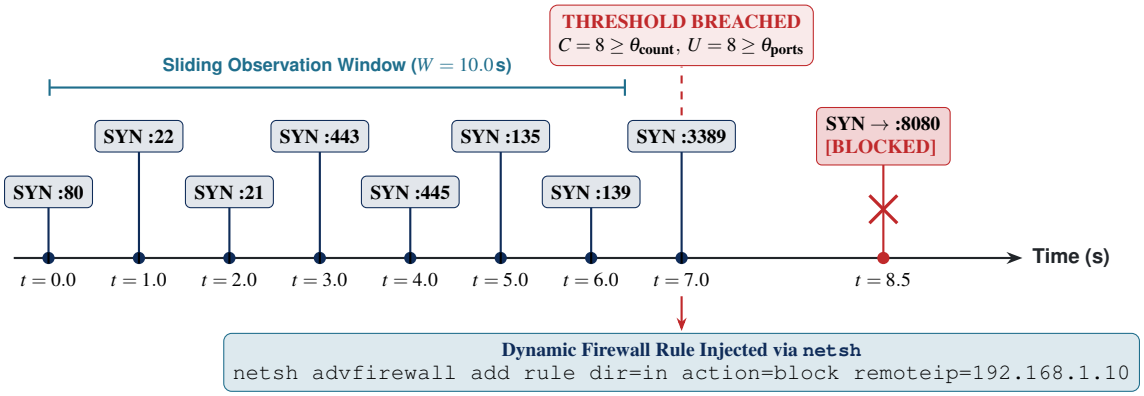


Figure 5: Timeline of Sliding-Window Scan Detection and Dynamic Firewall Rule Injection

4 Binary Packet and Frame Structure

4.1 Layer 2: Ethernet II Frame Header

Every transmitted probe is encapsulated in an Ethernet II frame consisting of 14 header octets:



Figure 6: Layer 2 Ethernet II Frame Structure (Total Header: 14 Octets)

4.2 Layer 3: IPv4 Packet Header

The IPv4 packet is constructed strictly according to RFC 791. Standard probes use a 20-byte base header without IP options:

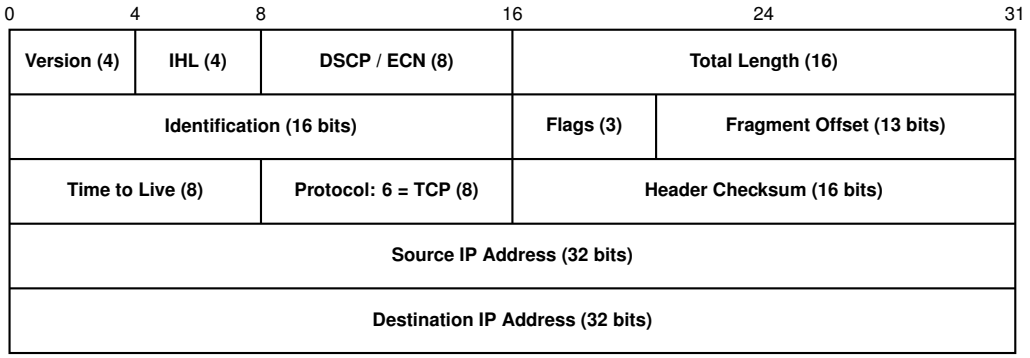


Figure 7: Layer 3 IPv4 Packet Header Format (20 Octets Base)

4.3 Layer 4: TCP Segment Header

The TCP header is serialized per RFC 793 with control flags explicitly set for the SYN half-open probe ($Flags = 0\text{x}02$):

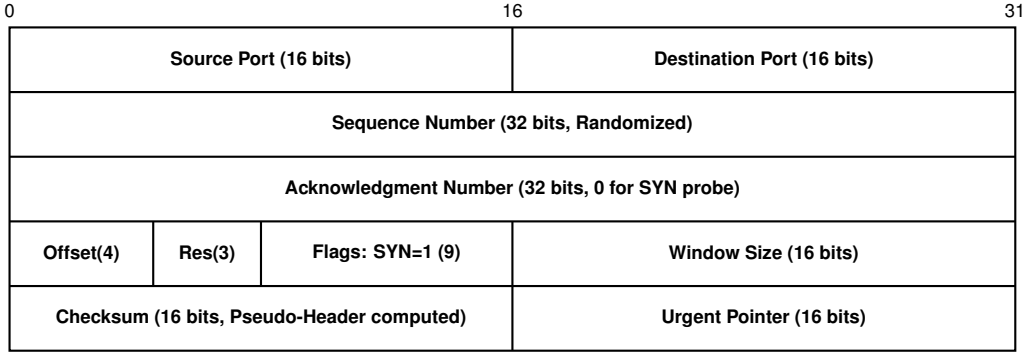


Figure 8: Layer 4 TCP Segment Header Format (20 Octets Base)

4.4 TCP Checksum Calculation and Pseudo-Header Construction

Under RFC 793, the TCP checksum is not calculated over the TCP segment alone. It covers a 12-byte pseudo-header prepended to the TCP segment to verify that the packet reached its intended destination IP address:

$$\text{Checksum} = \sim \left(\sum_{i=1}^{N} W_i \pmod{2^{16} - 1} \right) \tag{1}$$

where W_i represents consecutive 16-bit words across the concatenated pseudo-header and TCP segment. The pseudo-header structure is defined in Table 3:

Table 3: IPv4 TCP Pseudo-Header Field Layout for Checksum Calculation

Byte Offset	Field Name	Value in SYN Probe
0–3	Source IPv4 Address	Attacker IPv4 (e.g., `192.168.1.10`)
4–7	Destination IPv4 Address	Target IPv4 (e.g., `192.168.1.25`)
8	Reserved Zero Octet	`0x00`
9	Protocol Identifier	`0x06` (TCP Transport Protocol)
10–11	TCP Segment Length	Length of TCP header + options + payload (e.g., 20)

4.5 OS Fingerprinting Response Feature Matrix

The fingerprinting engine parses incoming SYN/ACK frames and extracts four diagnostic attributes to compute OS classification scores:

Table 4: TCP/IP Stack Fingerprinting Signatures Extracted from SYN/ACK Packets

Diagnostic Feature	Packet Field	Windows 10/11	Linux 5.x / 6.x	macOS / BSD
Initial TTL	IPv4 TTL	128	64	64
TCP Window Size	TCP Window	64240 or 65535	29200 or 65160	65535
MSS Option	TCP Option 2	1460	1460	1460
Window Scale Option	TCP Option 3	8	7	6
TCP Timestamp	TCP Option 8	Often Absent	Typically Present	Present
Option Sequence	TCP Options	MSS, NOP, WS, NOP, NOP, SACK	MSS, SACK, TS, NOP, WS	MSS, NOP, WS, NOP, NOP, TS

5 Implementation Plan and Modular Architecture

5.1 Modular Repository Architecture

The project codebase is organized into four decoupled packages:

- **`common/`:** Shared core networking utilities.
  - `packet.py`: Implements `build_syn_frame()`, `parse_ipv4_tcp_frame()`, and the RFC 1071 checksum routine.
  - `npcap.py`: Provides native `ctypes` wrappers for `wpcap.dll` functions, including `pcap_open`, `pcap_sendpacket`, and `pcap_next_ex`.
- **`attacker/`:** Multi-phase attack orchestration suite.
  - `main.py`: Master CLI coordinator linking all reconnaissance and exploit phases.
  - `scanner.py`: TCP SYN port scanner implementing the half-open probe loop.
  - `banner.py`: Stream socket connection routines for passive and active banner extraction.
  - `fingerprint.py`: Heuristic scorer comparing SYN/ACK attributes against known signatures.
  - `exploits/`: Dedicated modules for anonymous FTP authentication and HTTP directory path traversal.
- **`victim/`:** Reproducible demonstration targets.
  - `ftp_server.py`: Multithreaded Python FTP server configured to accept anonymous logins.
  - `http_server.py`: HTTP daemon containing an intentional path traversal flaw.
- **`defense/`:** Countermeasure implementation.
  - `detector.py`: In-memory sliding-window packet queue tracking per-IP probe frequency and destination port counts.
  - `main.py`: Automated alert dispatcher executing `netsh advfirewall` commands to enforce temporary IP blocks.

6 Justification and Technical Viability

The technical approach relies on three standard protocol behaviors:

1. **Deterministic Kernel Responses:** RFC 793 specifies that an operating system must reply with RST for packets to closed ports and SYN/ACK for active listening ports. Because these replies come directly from the kernel network stack, the scanner determines port states without establishing an application session.
2. **Link-Layer Transmission via Npcap:** Standard Windows sockets (`SOCK_RAW`) restrict raw TCP packet construction for unprivileged users. Interfacing directly with the Npcap driver via `ctypes` bypasses transport-layer limits and transmits custom Ethernet frames onto the local wire.
3. **Vendor Divergence in TCP Defaults:** Operating system vendors configure distinct default values for initial TTL, TCP buffer window size, and option negotiation sequences. Windows defaults to an initial TTL of 128, whereas Linux uses 64. Across a local subnet without routing hops, these signatures distinguish the target operating system reliably.

6.1 Operational Resilience and Failure Handling

Beyond protocol viability, the system handles operational faults through explicit state tracking:

- **Asynchronous Probe Timeouts:** Network operations enforce a 1.5-second per-probe timeout so that dropped or filtered packets do not stall scanner thread execution.
- **Automated Firewall Rule Cleanup:** Dynamic firewall rules added via `netsh` are tracked in memory and removed upon exit, preventing persistent network lockouts.
- **Sandboxed Demonstration Services:** Target services run inside dedicated unprivileged test directories to keep proof-of-concept verification isolated from host files.

7 Expected Outcomes of a Successful Attack

7.1 Evaluation Criteria

Table 5 summarizes the target criteria for evaluating each attack phase:

Table 5: Evaluation Metrics and Validation Methods for Attack Phases

Phase	Evaluation Target	Success Indicator	Validation Method
1. Port Scan	Accurate port classification	Port states match host reality (Open, Closed, Filtered).	Compare scanner output with `netstat -an` on victim host.
2. Banners	Software identification	Extracted string contains correct daemon name and version.	Match against service launch configurations and headers.
3. OS Fingerprint	OS family classification	Target identified as Windows with confidence $\geq 80\%$.	Compare against victim system properties (`winver`).
4. Exploits	Access compromise	Unauthorized read/access on target services demonstrated.	Session logs confirm anonymous FTP authentication and HTTP directory path traversal read.

7.2 Anticipated Execution Telemetry and Verification Output

To verify pipeline behavior against its functional design criteria, the anticipated console output of an automated reconnaissance and exploit cycle is shown below. This trace details the data schema, heuristic classifications, and exploit verification results:

Attacker Console Output: Benchmark Execution Trace

```
============================================================
  CUSTOM PORT SCANNER: ATTACK TOOLKIT
============================================================
  Target       : 192.168.0.116
  Attacker     : 192.168.0.105 (a0:29:42:92:ef:19)
  Target MAC   : d8:5e:d3:99:09:6c
  Ports        : [21, 80, 443, 445]
  Scan delay   : 0.5s

  [PHASE 1] TCP SYN Port Scanning
  ------------------------------------------------------------
     21  OPEN                           TTL=128 WIN=65535
     80  OPEN                           TTL=128 WIN=65535
    443  NO RESPONSE / POSSIBLY FILTERED
    445  OPEN                           TTL=128 WIN=65535

  [PHASE 2] Service Banner Grabbing
  ------------------------------------------------------------
  [+]  21  FTP   -> LabFTP Server (Vulnerable Demo) Ready
  [+]  80  HTTP  -> VulnHTTP/1.0 (Lab Demo)

  [PHASE 3] OS Fingerprinting
  ------------------------------------------------------------
  Likely OS  : Windows family (Confidence: 89%)
  Version    : Windows 10/11 family
  Evidence   : Initial TTL=128, Window=65535, MSS=1460

  [PHASE 5] Targeted Exploit Execution
  ------------------------------------------------------------
  [EXPLOIT 1] FTP Anonymous Authentication (Port 21)
  Status     : SUCCESS (Authenticated as anonymous:guest)
  Directory  : 257 "/" | Files: drwxr-xr-x 1 ftp ftp 0 pub

  [EXPLOIT 2] HTTP Directory Path Traversal (Port 80)
  Status     : SUCCESS (Payload: ..\..\..\windows\win.ini)
  Payloads   : 12 attempted, 8 successful variations
  Disclosed  : [fonts] Arial=arial.ttf (arbitrary host file read)

============================================================
  ATTACK SUMMARY
============================================================
  Ports scanned : 4 | Open: 3 | Filtered: 1
  Banners found : 2/2 | OS guess: Windows family (89%)
  Exploits      : 2/2 successful (FTP Anonymous + HTTP Traversal)
============================================================
```

8 Defense Mechanism and Dynamic Countermeasures

8.1 Sliding-Window SYN Rate and Diversity Detection

Port scanning generates a characteristic footprint: multiple SYN packets directed at different destination ports within a short interval. The defensive detector monitors incoming TCP traffic using a two-part sliding window:

Detection Algorithm Formulation

For each unique source IP S, maintain a sliding event queue:

$$Q_S = \{(t_i, p_i) \mid t_i > t_{\text{current}} - W\}$$

where W is the sliding window duration ($W = 10.0$ seconds). On arrival of any packet with SYN $= 1$ and ACK $= 0$:

1. Append $(t_{\text{current}}, p_{\text{dest}})$ to Q_S.
2. Evict all entries where $t_i \leq t_{\text{current}} - W$.
3. Compute total events $C = |Q_S|$ and unique ports $U = |\{p_i \mid (\cdot, p_i) \in Q_S\}|$.
4. Trigger alert if $C \geq \theta_{\text{count}}$ (8) **and** $U \geq \theta_{\text{ports}}$ (5).

Checking both packet rate and port diversity prevents false positives from single-port bursts (such as parallel HTTP asset downloads) while catching multi-port scans.

8.2 Dynamic Host Firewall Mitigation

When the monitor triggers an alert and mitigation is enabled, it adds an inbound blocking rule through the Windows Advanced Firewall:

Windows Advanced Firewall Dynamic Block Rule Command

```
netsh advfirewall firewall add rule name="CustomPortScanBlock_192_168_1_10" ^
    dir=in action=block remoteip=192.168.1.10 protocol=TCP profile=any
```

The defense process records active rules in memory and removes them upon exit to prevent persistent network misconfiguration.

8.3 Security Analysis and Limitations

- **False-Positive Suppression:** Requiring destination port diversity distinguishes scans from high-throughput single-port traffic, such as web browsing or database connection pools.
- **Sub-Second Mitigation:** Rule generation via `netsh` applies within milliseconds of crossing the detection threshold.
- **Known Limitations:** Slow scans with probe intervals longer than the 10-second window will evade detection. Attackers could also bypass the single-IP filter by distributing probes across multiple addresses.

References

[1] Postel, J., "Transmission Control Protocol," *RFC 793*, Defense Advanced Research Projects Agency, 1981.
[2] Postel, J., "Internet Protocol," *RFC 791*, Defense Advanced Research Projects Agency, 1981.
[3] Braden, R., Borman, D., and Partridge, C., "Computing the Internet Checksum," *RFC 1071*, 1988.
[4] Fyodor (Lyon, G.), "Remote OS Detection via TCP/IP Stack Fingerprinting," *Phrack Magazine*, vol. 8, no. 54, 1998.
[5] MITRE ATT&CK, "Network Service Scanning (T1046)," *MITRE Corporation*, 2023. https://attack.mitre.org/techniques/T1046/.
[6] National Vulnerability Database, "CVE-2021-41773: Apache HTTP Server Path Traversal Vulnerability," *NIST*, 2021. https://nvd.nist.gov/vuln/detail/CVE-2021-41773.
[7] Nmap Project, "Npcap: Windows Packet Capture Library & Driver," *Npcap Reference Documentation*, 2024. https://npcap.com/.